

```
# Uninstall the currently installed version of the Transformers library.
# This helps avoid version conflicts with the version we want to install.
!pip uninstall -y transformers

# Install specific versions of the required libraries.
!pip install -U transformers==4.46.3 \
    accelerate \
    sentencepiece \
    huggingface-hub==0.26.2
```

```
Found existing installation: transformers 4.46.3
Uninstalling transformers-4.46.3:
  Successfully uninstalled transformers-4.46.3
Collecting transformers==4.46.3
  Using cached transformers-4.46.3-py3-none-any.whl.metadata (44 kB)
Requirement already satisfied: accelerate in /usr/local/lib/python3.12/dist-packages (1.14.0)
Requirement already satisfied: sentencepiece in /usr/local/lib/python3.12/dist-packages (0.2.2)
Requirement already satisfied: huggingface-hub==0.26.2 in /usr/local/lib/python3.12/dist-packages (0.26.2)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (3.29.4)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (26.2)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (6.0.3)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (2025.11)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (2.32.4)
Requirement already satisfied: tokenizers<0.21,>=0.20 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (0.19.1)
Requirement already satisfied: safetensors>=0.4.1 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (0.8.0)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers==4.46.3) (4.67.3)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub==0.26.2) (2025.10.2)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub==0.26.2) (4.12.2)
Requirement already satisfied: psutil in /usr/local/lib/python3.12/dist-packages (from accelerate) (5.9.5)
Requirement already satisfied: torch>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from accelerate) (2.11.0+cu128)
Requirement already satisfied: setuptools<82 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (3.6.1)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (3.1.6)
Requirement already satisfied: cuda-toolkit==12.8.1 in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,cudart,
Requirement already satisfied: cuda-bindings<13,>=12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelera
Requirement already satisfied: nvidia-cudnn-cu12==9.19.0.56 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accel
Requirement already satisfied: nvidia-cusparse-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->acce
Requirement already satisfied: nvidia-nccl-cu12==2.28.9 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerat
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->acceler
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from torch>=2.0.0->accelerate) (3.6.0)
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cubl
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cubl
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cubl
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[c
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cub
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[c
Requirement already satisfied: nvidia-cusparselt-cu12==12.5.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[c
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cu
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[c
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90.* in /usr/local/lib/python3.12/dist-packages (from cuda-toolkit[cublas,
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->transformers=
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->transformers==4.46.3) (3.
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->transformers==4.46.
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->transformers==4.46.
Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.12/dist-packages (from cuda-bindings<13,>=12.9.4->
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=2.0.0->
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->torch>=2.0.0->accelerate)
Using cached transformers-4.46.3-py3-none-any.whl (10.0 MB)
Installing collected packages: transformers
Successfully installed transformers-4.46.3
```

```
# Import the pytorch library for tensor operations and deep learning
import torch

# Import Model and tokenizer and pipeline from transformers library
from transformers import AutoModelForCausalLM, AutoTokenizer, pipeline

# Load model(Phi-3) and tokenizer
tokenizer = AutoTokenizer.from_pretrained("microsoft/Phi-3-mini-4k-instruct")
model = AutoModelForCausalLM.from_pretrained(
    "microsoft/Phi-3-mini-4k-instruct",
    # Automatically selects the available device CPU or GPU
    # GPU executes and loads the data faster than the CPU
    device_map="auto",
    #Automatically select the appropriate data type
    torch_dtype="auto",
    # Allow the model to run its own code
    trust_remote_code=
```

```

True
,
)
# Create a pipeline
generator = pipeline(
    "text-generation", # tells the pipeline that the model to generate text
    model=model, # previously loaded model
    tokenizer=tokenizer, # previously loaded tokenizer
    # return the only generated text not the original prompt
    return_full_text=
    False
,
# Generates a maximum of 50 new tokens
max_new_tokens=50,
# Disable random sampling ,so the model can mostly generate the next coming token not random
do_sample=
False
,
)

```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF\_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.

```

warnings.warn(
tokenizer_config.json:      3.44k/? [00:00<00:00, 276kB/s]

tokenizer.model: 100%                               500k/500k [00:00<00:00, 26.9MB/s]

tokenizer.json:      1.94M/? [00:00<00:00, 69.4MB/s]

added_tokens.json: 100%                               306/306 [00:00<00:00, 35.4kB/s]

special_tokens_map.json: 100%                         599/599 [00:00<00:00, 68.2kB/s]

config.json: 100%                                     967/967 [00:00<00:00, 80.8kB/s]

configuration_phi3.py: 11.2k/? [00:00<00:00, 723kB/s]
A new version of the following files was downloaded from https://huggingface.co/microsoft/Phi-3-mini-4k-instruct:
- configuration_phi3.py
. Make sure to double-check they do not contain any added malicious code. To avoid downloading new versions of the code file, you
modeling_phi3.py:      73.2k/? [00:00<00:00, 4.49MB/s]
A new version of the following files was downloaded from https://huggingface.co/microsoft/Phi-3-mini-4k-instruct:
- modeling_phi3.py
. Make sure to double-check they do not contain any added malicious code. To avoid downloading new versions of the code file, you
WARNING:transformers_modules.microsoft.Phi-3-mini-4k-instruct.f39ac1d28e925b323eae81227eaba4464caced4e.modeling_phi3:`flash-atte
WARNING:transformers_modules.microsoft.Phi-3-mini-4k-instruct.f39ac1d28e925b323eae81227eaba4464caced4e.modeling_phi3:Current `fl
model.safetensors.index.json: 16.5k/? [00:00<00:00, 1.57MB/s]

Downloading shards: 100%                             2/2 [00:58<00:00, 28.44s/it]

model-00001-of-00002.safetensors: 100%                4.97G/4.97G [00:32<00:00, 326MB/s]

model-00002-of-00002.safetensors: 100%                2.67G/2.67G [00:25<00:00, 271MB/s]

Loading checkpoint shards: 100%                       2/2 [00:31<00:00, 14.94s/it]

generation_config.json: 100%                          181/181 [00:00<00:00, 12.3kB/s]

```

```

# Tell the model to perform a task using a variable
# This prompt tell the model what to generate
prompt = "Write an email apologizing to Sarah for the tragic gardening mishap. Explain how it happened."
# generates the output using the text generation pipeline
output = generator(prompt)
#output[0] accesses the first generated result.
# 'generated_text' retrieves the actual generated text.
print(output[0]['generated_text'])

```

Mention the steps you're taking to prevent it in the future.

Dear Sarah,

I hope this message finds you well. I am writing to express my sincerest apologies for the unfortunate incident that occurred

```

prompt = "The capital of France is"
# Tokenize the input prompt

```

```
# tokenizer convert the text into token Ids
# return_tensors="pt" returns the tokens as PyTorch tensors.
input_ids = tokenizer(prompt, return_tensors="pt").input_ids
# Move the token IDs to CPU
# use this if your device uses CPU as runtime input_ids = input_ids.to("cpu")
input_ids = input_ids.to("cuda")
# Pass the token IDs through the Transformer model.The model.model() function runs only the Transformer layers.
# This returns the hidden representations (embeddings) from the last Transformer layer.
# The LM Head is NOT applied yet.
model_output = model.model(input_ids)
#lm_head =LM head is the final layer in the transformer block .It job is to take the hidden representaations .and generate scor
# model_output= contain the hidden representations (embeddings)
# model.lm_head it predicts the next token
lm_head_output = model.lm_head(model_output[0])
```

```
#select the scores of first and last input token
# Find the token ID with the highest relevance score using argmax().
# argmax(-1) it returns the index of tokens
token_id = lm_head_output[0,-1].argmax(-1)
# tokenizer write the tokens back into text
tokenizer.decode(token_id)
```

```
'Paris'
```

```
#Shows the shape
model_output[0].shape
```

```
torch.Size([1, 5, 3072])
```

```
# Shows the shape
lm_head_output.shape
```

```
torch.Size([1, 5, 32064])
```

```
# Prompt tells the model to perform a task based in the prompt
prompt = "Write a very long email apologizing to Sarah for the tragic gardening mishap. Explain how it happened."
```

```
# Tokenizer convert the text into token IDs
input_ids = tokenizer(prompt, return_tensors="pt").input_ids
# Move the token IDs to CPU
input_ids = input_ids.to("cuda")
```

```
# Measure time how long it can take to execute the code
%%timeit -n 1
# Generate the text using the model
generation_output = model.generate(
# generated token IDs
input_ids=input_ids,
# Generate max 50 new tokens
max_new_tokens=100,
#True = Model stores the previous calculations and reuse them
use_cache=
True
)
```

```
4.92 s ± 296 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

```
#Measure time how long it can take to execute the code once
%%timeit -n 1
# Generate the text using the mdoel
generation_output = model.generate(
# Previously generated tokens
input_ids=input_ids,
# Generate the max 100 new tokens
max_new_tokens=50,
# False =Model recalculates everything from the begining everytime it generate the tokens
use_cache=
False
)
```

```
13.9 s ± 520 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

Start coding or [generate](#) with AI.